量子计算物流优化挑战赛题目解析 - Google Gemini

Date: April 17, 2026 at 01:56 PM
Turns: 4
Source: Gemini Chat

Turn 1

🤖 Assistant

为了让你直观理解如何将实际问题转化为 QUBO 模型，我们以物流优化中最基础的
旅行商问题（TSP）为例。
场景设定：
配送员需要从仓库 (点0) 出发，访问 **2个客户点 (点1, 点2)**，最后回到**仓库**。要求路径最短且每个点只去一次。
各点之间的距离（对称）：$d_{01} = 2, d_{02} = 3, d_{12} = 1$。

第一步：定义决策变量 (Binary Variables)

我们需要确定在第 t 个次序访问了哪一个点。
定义 $x_{i,t}$：如果在第 t 步访问了点 i，则 $x_{i,t} = 1$；否则为 0。
在这个小例子中，$i \in \{1, 2\}$，$t \in \{1, 2\}$（仓库是起点和终点，是固定的，不需要变量）。
变量列表：$x_{1,1}, x_{1,2}, x_{2,1}, x_{2,2}$。共有 4 个变量。

第二步：构建目标函数 (H_{obj})

我们要最小化总距离。总路径为：仓库 $\rightarrow$ 第1步的点 $\rightarrow$ 第2步的点 $\rightarrow$ 仓库。

$$H_{obj} = \sum_{i=1}^{2} d_{0i} x_{i,1} + d_{12}(x_{1,1}x_{2,2} + x_{2,1}x_{1,2}) + \sum_{i=1}^{2} d_{i0} x_{i,2}$$

第一项是仓库到第一站，第二项是站与站之间，第三项是最后一站回仓库。

第三步：构建约束条件的罚函数 ($H_{penalty}$)

QUBO 不允许有硬性约束，所以我们要把约束转换成惩罚项。

1. **约束 1：每个点必须访问一次**
   对于点 1：$(x_{1,1} + x_{1,2} - 1)^2 = 0$
   对于点 2：$(x_{2,1} + x_{2,2} - 1)^2 = 0$
2. **约束 2：每个时间点只能在一个位置**
   时间 1：$(x_{1,1} + x_{2,1} - 1)^2 = 0$
   时间 2：$(x_{1,2} + x_{2,2} - 1)^2 = 0$
   将这些项展开（记得 $x^2 = x$），并乘上一个足够大的惩罚系数 P（比如 $P = 10$）。

第四步：合并为 Q 矩阵

最终的 QUBO 模型为：$H = H_{obj} + P \cdot (H_{penalty1} + H_{penalty2})$。
我们将所有项合并，整理成 $x^T Q x$ 的形式。
假设我们排列变量的顺序为 $[x_{1,1}, x_{1,2}, x_{2,1}, x_{2,2}]$，得到的 Q 矩阵（对角线是线性项系数，非对角线是交互项系数）如下表所示（仅为逻辑示意）：

变量	x1,1	x1,2	x2,1	x2,2
x1,1	d01 −2P	2P	2P	d12
x1,2		d10 −2P	d12	2P
x2,1			d02 −2P	2P

变量	x1,1	x1,2	x2,1	x2,2
x2,2				d20 −2P
注意： 实际计算时，(x1+x2−1)2=x1+x2+2x1x2−2x1−2x2+1。常数项 1 对寻找最优解无影响，可以直接舍去。				

第五步：调用算法求解 (Kaiwu SDK)

在 MathorCup 比赛中，你不需要手动算这个矩阵。你的代码逻辑应该是：

```
# 伪代码示例
from kaiwu_sdk import QuboSolver

# 你的 Q 矩阵（二维数组或字典形式）
matrix_Q = { (0,0): -18, (0,1): 20, ... }

solver = QuboSolver()
result = solver.solve(matrix_Q) # 发送给量子/类量子后端
print(result.best_binary_config) # 输出 [1, 0, 0, 1] 这种结果
```

💡 比赛提分技巧

- **惩罚系数调优：** 这是最难的一步。如果结果里点 1 被去了两次，说明 P 太小；如果结果路径非常长但满足约束，说明 P 太大压制了距离优化。
- **松弛变量：** 题目 A 涉及到“时间窗”和“载重”，这些是不等式（如 $\sum w \leq C$）。你需要查阅资料，学习如何引入 **Slack Variables** 将不等式转为 QUBO。
- **预处理：** 如果点 A 到点 B 距离无限远，直接在 Q 矩阵里给 $x_A x_B$ 设一个极大的惩罚值，相当于“剪枝”。

Turn 2

🤖 Assistant

非常有道理！直接给结果确实太“跳跃”了。在数学建模竞赛中，推导过程才是灵魂。
我们现在把这个 4 变量的 TSP 问题像拆解闹钟一样，一步步拆开来看。

第一步：拆解罚函数 $H_{penalty}$

我们有两个约束，每个约束都要平方展开。

1. 约束 1：每个点必须访问一次（点约束）

对点 1：$P(x_{1,1} + x_{1,2} - 1)^2$
展开：

$$P(x_{1,1}^2 + x_{1,2}^2 + 1 + 2x_{1,1}x_{1,2} - 2x_{1,1} - 2x_{1,2})$$

利用 $x^2 = x$（二值变量特性）简化：

$$P(x_{1,1} + x_{1,2} + 1 + 2x_{1,1}x_{1,2} - 2x_{1,1} - 2x_{1,2})$$

合并同类项（$x_{1,1} - 2x_{1,1} = -x_{1,1}$）：

$$P(-x_{1,1} - x_{1,2} + 2x_{1,1}x_{1,2} + 1)$$

同理，对点 2 的展开结果为：

$$P(-x_{2,1} - x_{2,2} + 2x_{2,1}x_{2,2} + 1)$$

2. 约束 2：每个时间点只能在一个位置（时间约束）

对时间 1：$P(x_{1,1} + x_{2,1} - 1)^2$
展开并化简（过程同上）：

$$P(-x_{1,1} - x_{2,1} + 2x_{1,1}x_{2,1} + 1)$$

对时间 2：

$$P(-x_{1,2} - x_{2,2} + 2x_{1,2}x_{2,2} + 1)$$

第二步：列出目标函数 H_{obj}

目标函数本来就是展开的：

$$H_{obj} = d_{01}x_{1,1} + d_{02}x_{2,1} + d_{12}x_{1,1}x_{2,2} + d_{21}x_{2,1}x_{1,2} + d_{10}x_{1,2} + d_{20}x_{2,2}$$

第三步：合并所有项（大总和）

现在我们把 $H_{obj} + H_{p1} + H_{p2}$ 全部加起来，按**变量类型**分类：

1. 线性项（只含一个 x 的项，决定对角线系数）

看 $x_{1,1}$ 的系数：

- 来自 H_{obj}：d_{01}
- 来自点约束 1：$-P$
- 来自时间约束 1：$-P$
- **总和：(d01−2P)**
  看 $x_{1,2}$ 的系数：
- 来自 H_{obj}：d_{10}
- 来自点约束 1：$-P$
- 来自时间约束 2：$-P$
- **总和：(d10−2P)**
  （其余变量同理可推）

2. 交互项（含两个 x 相乘的项，决定非对角线系数）

- **互斥产生的项（罚金）：**
  - $x_{1,1}x_{1,2}$（同点不同时）：系数为 $2P$
  - $x_{1,1}x_{2,1}$（同时不同点）：系数为 $2P$
- **路径产生的项（距离）：**
  - $x_{1,1}x_{2,2}$（第1步去点1，第2步去点2）：系数为 d_{12}

- $x_{2,1}x_{1,2}$（第1步去点2，第2步去点1）：系数为 d_{21}

第四步：填入 Q 矩阵

现在这个表就不再是“玄学”了，每一项都有出处：

变量	x1,1	x1,2	x2,1	x2,2
x1,1	d01−2P	2P (互斥罚)	2P (互斥罚)	d12 (路径长)
x1,2		d10−2P	d21 (路径长)	2P (互斥罚)
x2,1			d02−2P	2P (互斥罚)
x2,2				d20−2P

总结公式逻辑

这个过程可以用一个简单的口诀来记：

1. **对角线：** 原始路程成本，**减去** 它参与的所有约束的 P 次数（通常每个变量参与两个约束，所以是 $-2P$）。
2. **非对角线：**

- 如果两个变量**冲突**（不能同时选），填 **2P**。
- 如果两个变量**衔接**（走完 A 走 B），填 **距离 d**。
- 其他不相关的，填 **0**。
  这个展开过程是不是清晰多了？你现在能看出来为什么 $x_{1,1}x_{2,2}$ 位置填的是距离 d_{12} 而不是 $2P$ 了吗？

Turn 3

🤖 Assistant

为了将所有的展开项整理成标准矩阵形式

$$H(x) = x^T Q x$$

，我们需要把决策变量排列成一个向量，并将每一项的系数精确填入矩阵 Q。在 QUBO 问题中，Q 矩阵通常约定为 **上三角矩阵**。
以下是针对我们讨论的 4 变量 TSP 案例的最终整理过程：

1. 定义决策向量 x

我们将之前的 4 个二值变量按顺序排列为一个列向量：

$$x = \begin{bmatrix} x_{1,1} \\ x_{1,2} \\ x_{2,1} \\ x_{2,2} \end{bmatrix}$$

2. 构建 Q 矩阵

在 Q 矩阵中，**线性项**（单个 x 的系数）填在**对角线**，**交互项**（两个 x 相乘的系数）填在**非对角线**的对应位置（上三角部分）：

$$Q = \begin{bmatrix} Q_{11} & Q_{12} & Q_{13} & Q_{14} \\ 0 & Q_{22} & Q_{23} & Q_{24} \\ 0 & 0 & Q_{33} & Q_{34} \\ 0 & 0 & 0 & Q_{44} \end{bmatrix}$$

代入之前详细展开的各项系数，得到最终的 **Q 矩阵**：

$$Q = \begin{bmatrix} (d_{01} - 2P) & 2P & 2P & d_{12} \\ 0 & (d_{10} - 2P) & d_{21} & 2P \\ 0 & 0 & (d_{02} - 2P) & 2P \\ 0 & 0 & 0 & (d_{20} - 2P) \end{bmatrix}$$

3. 数学验证：为什么这样整理？

当你执行矩阵乘法 $H(x) = x^T Q x$ 时，其展开式为：

$$H(x) = \sum_i Q_{ii} x_i^2 + \sum_{i<j} Q_{ij} x_i x_j$$

- **对角线项：** 由于 $x_i \in \{0, 1\}$，根据二值变量特性 $x_i^2 = x_i$。因此对角线上的 $(d_{01} - 2P)x_{1,1}^2$ 变成了 $(d_{01} - 2P)x_{1,1}$，这完美对应了展开式中带有距离和惩罚的**线性部分**。

- **非对角线项：** 如 $Q_{12}x_{1,1}x_{1,2}$ 对应了 $2Px_{1,1}x_{1,2}$（互斥惩罚），而 $Q_{14}x_{1,1}x_{2,2}$ 对应了 $d_{12}x_{1,1}x_{2,2}$（路径衔接成本）。

4. 关键填数准则（解题核心）

在 MathorCup 编程阶段，你只需要按照以下逻辑往矩阵里“填数”：

- **准则 1（对角线 Qii）：** 如果 $x_i = 1$ 时它本身自带的“成本”（如运输时间）是多少？加上它 。然后，它参与了多少个“必须选一个”的约束？参与一次，就 **减去** 一个 P。
- **准则 2（非对角线 Qij）：**
  - 如果 x_i 和 x_j**不能同时选**（互斥）：填 **2P**。
  - 如果 x_i 和 x_j**同时选代表走过一段路**（衔接）：填该段路的 **时间/距离 d**。
  - **其他情况：** 填 **0**。

💡 为什么 Kaiwu SDK 需要这个形式?

量子计算后端（如 CIM 或量子退火机）本质上是一个物理系统，矩阵中的 Q_{ij} 对应的是量子比特（或光脉冲）之间的**耦合强度**。你把这个矩阵传过去，它就在物理层面上模拟这些吸引力和排斥力，最后系统会自发趋于能量最低的状态，那个状态对应的 x 向量就是你的最优配送方案 。
这就是从物理场景到数学矩阵，再到量子求解的完整逻辑闭环 。在应对 MathorCup A 题时，最难的往往不是这个公式，而是如何处理更复杂的**不等式约束**（如载重限制），你需要聊聊那个吗?

Turn 4

🤖 Assistant

既然你已经彻底理解了 $H(x)$ 的推导和 Q 矩阵的构造，那么恭喜你，你已经完成了**“从物理问题到数学模型”**最艰难的跨越。
接下来的核心任务就是“求解”**，即寻找那一组二值向量 $x = [x_1, x_2, \ldots, x_n]$，使得 $H(x) = x^T Q x$ 达到全局最小值 。
由于这是一个 **NP-Hard** 问题，对于大规模变量，我们无法通过暴力穷举来解决。在 MathorCup 比赛中，通常有以下几种求解路径：

1. 经典启发式算法（用于本地测试与基准比对）

在没有量子硬件的情况下，我们通常使用模拟物理过程的经典算法：

- **模拟退火 (Simulated Annealing, SA)**：
  - **原理**：模拟固体退火过程。初始时给系统一个高“温度”，让它有概率跳出局部最优解（哪怕这个解暂时让 $H(x)$ 变大），随着温度降低，系统逐渐稳定在能量最低点 。
  - **优点**：实现简单，是比赛中最常用的 Baseline 。
- **禁忌搜索 (Tabu Search)**：
  - **原理**：通过维护一个“禁忌表”来记录已经搜索过的区域，强制算法去探索未知的解空间，防止陷入局部循环 。

2. 量子/类量子求解（本次竞赛的核心要求）

题目明确提到了 **Kaiwu SDK** 和 **相干辛机 (CIM)**：

- **相干辛机 (CIM)**：
  - **原理**：利用光学脉冲的相位来模拟 Ising 模型的自旋状态 。它利用激光脉冲在光纤环路中的相干叠加，通过物理演化自动寻找最低能量态（即 $H(x)$ 最小） 。
- **Kaiwu SDK**：
  - **操作流**：腾讯提供的这个 SDK 封装了高效的后端求解器（包括 CIM 模拟器和 FPGA 加速器）。你只需要通过 API 提交你的 Q 矩阵，它会返回一个最优的 x 向量 。

3. 编程求解的具体步骤

以你已经构建好的 Q 矩阵为例，在代码层面通常遵循以下流程：

第一步：矩阵转化

由于 SDK 通常不直接接收二维数组，你需要将 Q 矩阵转化为其要求的格式（通常是**三元组字典**）：

```
# 示例：将 Q 矩阵转化为三元组字典 {(i, j): value}
qubo_dict = {}
for i in range(n):
    for j in range(i, n):  # 仅上三角部分 [cite: 61, 67]
        if Q[i, j] != 0:
            qubo_dict[(i, j)] = Q[i, j]
```

第二步：调用求解器

以 Kaiwu SDK 的逻辑为例：

```
from kaiwu_sdk import QuboSolver

# 实例化求解器
solver = QuboSolver()

# 提交任务，求解最小能量态 [cite: 71]
result = solver.solve(qubo_dict)

# 获取能量最小对应的 x 向量
best_x = result.best_binary_config  # 例如输出 [1, 0, 0, 1] [cite: 11]
min_energy = result.best_energy
```

第三步：结果解码

拿到 `best_x` 后，你需要根据你定义的变量顺序（如 $x_{1,1}, x_{1,2} \ldots$）将其还原为具体的路径方案 。

4. 影响求解效果的关键：参数调优

即使模型对了，算法也可能找不到最优解，你需要关注以下两个变量：

- **惩罚系数 P 的“甜点位”**：
  - 如果 P 设为 10^6，目标函数的微小优化会被庞大的惩罚项淹没，算法只会在满足约束的解里乱撞 。
  - 建议先运行一版，观察结果：如果违反约束，就调大 P；如果总时间降不下来，就调小 P。
- **求解步数与时间**：对于复杂问题，增加迭代次数或退火时长通常能显著提升找到全局最优解的概率。
  既然你已经掌握了 Q 的推导，那么在大规模物流场景下，变量数会随着点位增加呈平方级增长（例如 50 个点对应 $50^2 = 2500$ 个变量） 。面对这种规模的 Q 矩阵，你想了解如何通过“预剪枝”来降低计算维度吗？

Exported from Voyager
Generated on April 17, 2026 at 01:56 PM